Architecture and Model Research

AI-Powered Intelligent Query Resolution System

Infosys Springboard Virtual Internship 7.0 | Milestone 1

1. Project Overview

The project is a conversational platform that answers questions from uploaded knowledge documents. Users can upload PDF, DOCX, TXT, and FAQ CSV files and ask questions using text or voice. The system finds relevant document sections, generates an answer based on those sections, and shows the sources. It should work for different domains such as college handbooks, HR policies, and technical manuals.

2. Objectives

Build a document-based RAG pipeline; coordinate five specialized agents; support text and voice queries; show citations and confidence indicators; remember conversation context; and identify unanswered questions and knowledge gaps.

3. Proposed System Architecture

USER (text / voice in browser)
↓
FRONTEND (chat interface + Web Speech API)
↓
BACKEND API (Python + FastAPI) / Agent Orchestrator
↓
Query Understanding Agent → Clarification Agent (if needed)
↓
Retrieval Agent → Vector Database (ChromaDB)
↓
Response Generation Agent → Answer + source citations
↓
Conversation Memory Agent → Save context for follow-ups
↓
FRONTEND (answer, sources, confidence, optional speech)

Separate ingestion flow: Uploaded documents → Text extraction → Text chunks → Embedding model → ChromaDB vector index.

4. Technology and Model Research

Component	Proposed choice	Reason
Frontend	Streamlit or React	Simple chat and upload interface; React gives more browser control.
Backend	Python + FastAPI	Handles uploads, query processing, and agent logic.
Document extraction	PyMuPDF, python-docx, csv	Reads PDF, DOCX, TXT, and FAQ CSV files.

Embedding model	all-MiniLM-L6-v2 (Sentence Transformers)	Small, practical model for semantic similarity search.
Vector database	ChromaDB	Stores embeddings and supports similarity search.
Language model	Gemini API or an available open-source instruct model	Creates answers using retrieved document context.
RAG / agent tools	LangChain (optional)	Helps connect document retrieval, prompts, and agent steps.
Voice	Browser Web Speech API	Speech recognition and spoken answers in supported browsers.
Analytics	SQLite + simple charts	Tracks queries, confidence, and unanswered topics.

These are suggested implementation choices for research, not mandatory tools specified by the project brief. Model/API availability, pricing, privacy, and browser support should be checked before implementation.

5. How RAG Works

1. 1. User uploads documents; the system extracts their text.
2. 2. Text is divided into small, overlapping chunks and converted to embeddings (numeric representations).
3. 3. Embeddings and document metadata are stored in a vector database.
4. 4. A question is converted to an embedding; similar chunks are retrieved and ranked.
5. 5. The language model receives the question and retrieved chunks and writes an answer with citations.
6. 6. If evidence is weak, the system should say it cannot confirm the answer rather than invent information.

6. Five Agents and Their Roles

Query Understanding Agent

Recognizes whether the question is factual, procedural, comparative, or ambiguous.

Retrieval Agent

Searches the indexed knowledge base and ranks relevant chunks.

Response Generation Agent

Produces an understandable answer grounded in the retrieved chunks and includes source references.

Clarification Agent

Asks a follow-up question when a query is unclear or incomplete.

Conversation Memory Agent

Stores recent questions and answers so follow-up questions retain context.

7. Suggested Data Models

- Document: document_id, filename, file_type, upload_time, domain
- Chunk: chunk_id, document_id, text, page_or_section, embedding_id
- Conversation: conversation_id, user_id (optional), created_at
- Message: message_id, conversation_id, role, text, timestamp
- QueryResult: query_id, answer, cited_chunk_ids, relevance_scores, status

Confidence indicator: use a clearly labeled retrieval relevance measure, not an unsupported claim that the generated answer is certainly correct. Similarity scores may need normalization and evaluation before display.

8. Example Query Flow

A college uploads a student handbook. A user asks, “What is the attendance requirement?” The Retrieval Agent finds the attendance policy, the Response Generation Agent summarizes it, and the interface displays the handbook page as the source. If the user asks “What about exceptions?”, the Conversation Memory Agent uses the previous attendance context.

9. Testing and Milestone Plan

- Weeks 1–2: Research architecture and models; implement file ingestion, chunking, embeddings, and vector indexing; test retrieval on two domains.
- Weeks 3–4: Implement query classification, retrieval ranking, grounded answer generation, and agent orchestration.
- Weeks 5–6: Add clarification, conversation memory, voice interaction, and source transparency.
- Weeks 7–8: Add analytics and knowledge gap detection; test across at least three domains; refine and document results.

10. Evaluation Approach

Prepare test questions with known answers for each document collection. Check whether the correct source chunk appears in top search results, whether answers are supported by citations, whether ambiguous queries trigger clarification, and whether follow-ups preserve context. Also test unsupported questions, voice input, and document format handling.

11. Conclusion

This design combines semantic search, a language model, and five focused agents to make uploaded knowledge easier to access. A small working RAG prototype should be built first; voice, memory, transparency, and analytics can then be added in stages.

12. Development Approach: Agile and AWS

The internship provides AWS and Agile as possible approaches. They serve different purposes: Agile guides how the project is planned and developed, while AWS provides cloud services to host and run an application. The following is a proposed approach, not a requirement stated in the supplied project brief.

Option A: Agile Methodology (Recommended for Development)

Agile means building the project in small stages, testing each stage, collecting feedback, and improving it. It is suitable for this internship because the project already has four milestones across eight weeks.

- Iteration 1 (Weeks 1–2): Design architecture; upload PDF, DOCX, TXT, and CSV files; create chunks, embeddings, and a searchable vector index.
- Iteration 2 (Weeks 3–4): Implement query understanding, retrieval, answer generation, and agent routing.
- Iteration 3 (Weeks 5–6): Add clarification questions, conversation memory, voice input/output, and source display.
- Iteration 4 (Weeks 7–8): Add query analytics, test with three domains, fix issues, and prepare the demonstration.

Option B: AWS Cloud Deployment (Optional)

If the internship requires an AWS-based implementation, the same system could be deployed using the following services. These are possible choices and may involve cost or account configuration.

- Amazon S3: Store uploaded documents.
- Amazon EC2 or AWS App Runner: Host the Python FastAPI backend.
- Amazon Bedrock: Access supported foundation models for answer generation and potentially embeddings, depending on selected models and region.
- Amazon RDS or DynamoDB: Store document metadata, chat history, and query analytics.
- Vector search: Keep ChromaDB on the backend for a prototype, or evaluate an AWS-supported vector search service for production.

Recommended Project Setup

Use Agile as the development method. Build the initial version locally using Python, FastAPI, a simple web interface, sentence-transformer embeddings, ChromaDB, and a suitable language model. Add AWS deployment only if required by your mentor or project evaluation. This avoids unnecessary cloud setup during the first milestone.

13. Final Summary

The proposed AI-Powered Intelligent Query Resolution System uses RAG to retrieve relevant document passages and five agents to understand, search, answer, clarify, and remember queries. Agile provides a clear step-by-step implementation plan. AWS is an optional hosting and AI-service platform. The design supports the required text/voice interaction, source transparency, and future knowledge-gap analysis.